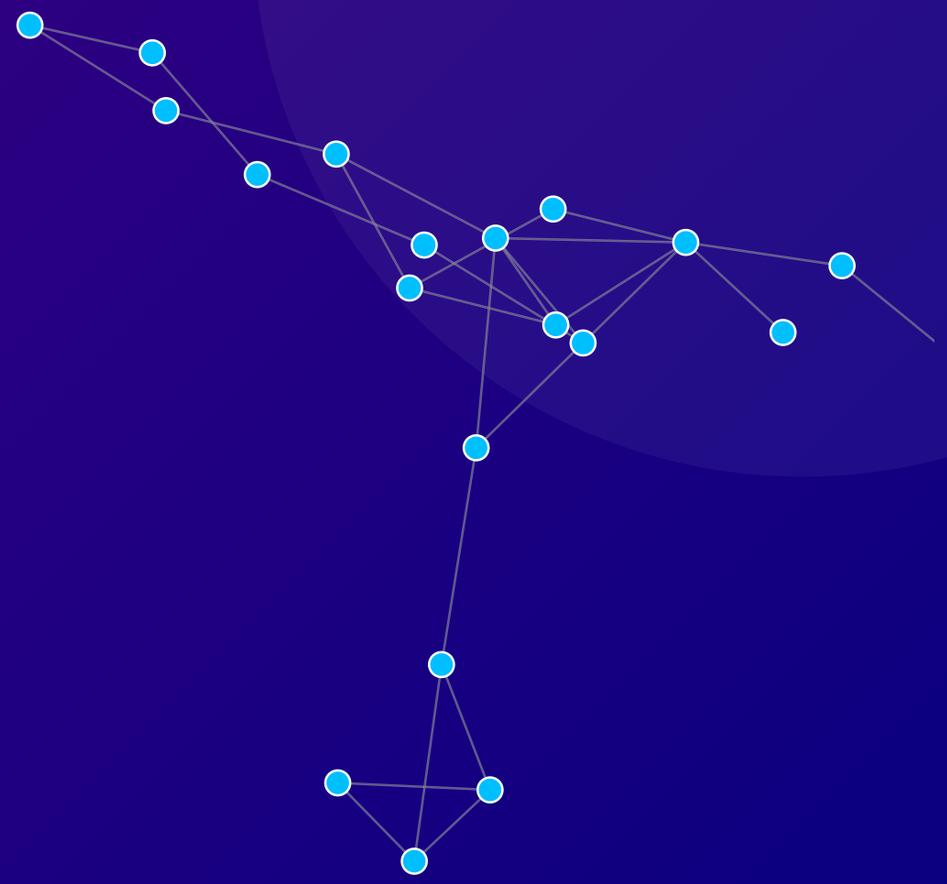
INDEX CONSTRUCTION & COMPRESSION

Week 4 | Information Retrieval

CS303X
Building Scalable Search Engines



Today's Agenda

01

Hardware Basics

Understanding I/O bottlenecks and memory constraints.

02

Index Construction Algorithms

BSBI (Block Sort-Based) and SPIMI (Single-Pass In-Memory).

03

Distributed Indexing

MapReduce framework and dynamic indexing strategies.

04

Index Compression

Dictionary compression and Postings compression (Gap Encoding).

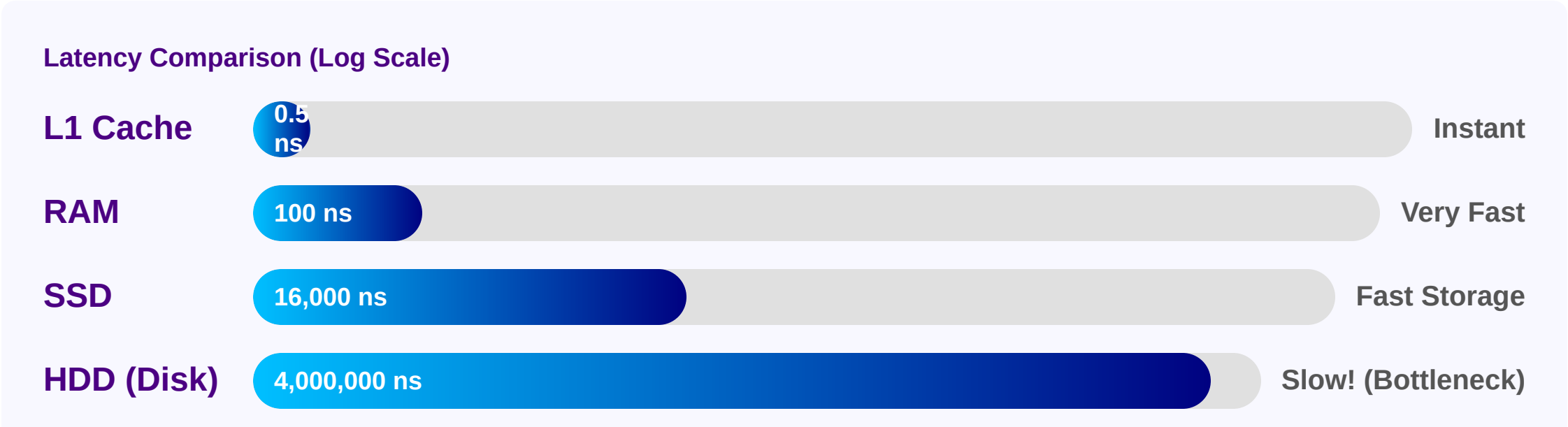
05

Query Processing

Phrase queries, wildcards, and spelling correction.

Hardware Basics: The I/O Bottleneck

- **Information Retrieval is I/O Bound:** Indexing speed is limited by how fast we can read/write to disk, not just CPU speed.
- **Disk Seek is Expensive:** Moving the disk head takes ~5-10ms. Random access is slow.
- **Sequential Access is Fast:** Always read large blocks of data to maximize throughput.



The I/O Bottleneck: Seek vs. Transfer

Seek Time

The time it takes for the disk head to move to the correct track and sector. This is mechanical overhead.

~5 ms (Slow!)

Transfer Time

The time to actually read the data once the head is positioned. This is electronic and fast.

~0.02 ms / KB

OPTIMIZATION STRATEGY: "MINIMIZE SEEKS"

- Batch Operations
- Use Large Buffers
- Compress Data

The Indexing Pipeline

Collection

Gathering raw documents (crawling, feeds).

`Raw HTML/PDF`

Tokenization

Breaking text into individual terms.

`Stream of Tokens`

Normalization

Case folding, stemming, stop words.

`Normalized Terms`

Indexing

Creating dictionary & postings lists.

`Inverted Index`

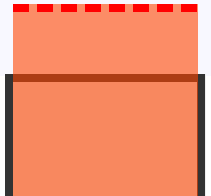
BSBI: Introduction

The Problem

We need to construct an inverted index for a massive collection (e.g., **100 GB** of text).

However, we only have limited RAM (e.g., **16 GB**).

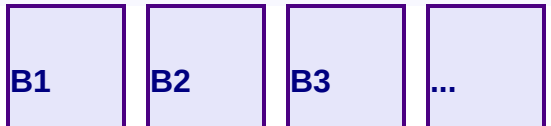
We **cannot hold all (Term, DocID) pairs in memory** to sort them.



The Solution

Block Sort-Based Indexing (BSBI) uses an external sorting algorithm.

1. Break the collection into equal-sized **blocks** that fit in memory.
2. Sort each block in memory.
3. Store intermediate sorted blocks on disk.
4. Merge them at the end.



BSBI Algorithm: The Process

1

Parse & Pair

Read documents stream. Extract **(TermID, DocID)** pairs sequentially.

2

Invert (Sort)

Accumulate pairs in RAM until a block is full. **Sort** pairs by TermID.

3

Write Segment

Write the sorted block to disk as an intermediate **Segment**.

4

Merge All

Read all segments simultaneously and **merge** into the final index.

Key Idea: Break the problem into chunks that fit in memory, then merge.

BSBI: Step-by-Step Example

1. INPUT DOCS

Doc 1
"Gladiator movie"

Doc 2
"Gladiator general"

MEMORY (BLOCK)

Accumulate Pairs

```
(gladiator, 1)
(movie, 1)
(gladiator, 2)
(general, 2)
```

↓ Sort (TermID)

```
(general, 2)
(gladiator, 1)
(gladiator, 2)
(movie, 1)
```

DISK

Segment 1
general: [2]
gladiator: [1, 2]
movie: [1]

Written sequentially

BSBI: Complexity Analysis

Time Complexity

$$O(T \log T)$$

- **T** = Total number of tokens (pairs).
- Sorting blocks is the dominant step.
- Merging is linear **O(T)**.
- Parsing is linear **O(T)**.

I/O Complexity

$$O(T)$$

- We read/write the data roughly **4 times**:
- 1. Read raw docs.
- 2. Write sorted segments.
- 3. Read segments for merge.
- 4. Write final index.

Key Constraint: During the merge phase, we must keep **N** file handles open (one per block). This limits scalability.

SPIMI: Single-Pass In-Memory Indexing

BSBI Limitation

BSBI generates millions of **(TermID, DocID)** pairs.

This wastes memory because the same TermID is repeated for every occurrence.

```
(gladiator, 1)
(gladiator, 2)
(gladiator, 3)
... repeats ...
```

Slow Sorting (O(N log N))

SPIMI Advantage

SPIMI builds the index **directly** in memory.

No intermediate pairs. We append DocIDs directly to the postings list.

gladiato	[1, 2, 3, ...]
movie	[1, 5, 8, ...]

Fast & Compact (O(N))

SPIMI Algorithm: The Process

1

Tokenize Stream

Process documents one by one. Extract tokens sequentially.

2

Lookup / Add

Check Dictionary (Hash Map). If term exists, get list. If not, add term.

3

Append DocID

Add DocID directly to the term's postings list in RAM. **No pairs!**

4

Sort & Write

When RAM full: Sort terms, write block to disk, clear RAM.

Key Difference from BSBI

SPIMI builds postings lists **directly** in memory. It saves memory by not storing redundant TermIDs in pairs.

SPIMI vs. BSBI: Comparison

FEATURE	BSBI (BLOCK SORT)	SPIMI (SINGLE PASS)
Data Structure	Stores (TermID, DocID) pairs. Wasteful.	Stores Postings Lists directly. Compact.
Sorting	Requires O(N log N) sort of pairs.	No sorting! Just append to lists (O(N)).
Dictionary	Needs global TermID mapping upfront.	Builds dictionary dynamically.
Memory Usage	High overhead for pair objects.	Efficient. Can process larger blocks.
Compression	Hard to compress pairs.	Can compress postings immediately.

Winner: SPIMI

SPIMI is generally preferred because it is faster, uses memory more efficiently, and doesn't require a pre

Distributed Indexing: Motivation

SINGLE MACHINE LIMITS

Storage Cap: Hard to fit PB of data.

Slow Indexing: Weeks to index the web.

Single Point of Failure: If it crashes, search dies.

DISTRIBUTED CLUSTER

Scalability: Add cheap nodes as needed.

Parallelism: Index in hours, not weeks.

Fault Tolerance: System survives node failures.

The Solution Framework: MapReduce (Google) / Hadoop

MapReduce Data Flow

Input Splits

Large files split into 16MB - 64MB chunks.

Raw Text

Map Phase

Parsers extract tokens. Outputs key-value pairs.

(Term, DocID)

Shuffle

System groups pairs by Key (TermID).

(Term, [List])

Reduce Phase

Merges lists into final Inverted Index.

Postings List

Key Insight: The **Shuffle** phase is the most expensive operation due to network bandwidth usage.

The Map Phase: Parsing & Pairing

INPUT SPLIT

Doc 1:
"Blue Sky"

MAP FUNCTION

worker_node.py

```
def map(docID, content):
  tokens = parse(content)
  for t in tokens:
    emit(t, docID)
```

KEY-VALUE PAIRS

```
("blue", 1)
("sky", 1)
...
```

Scale: This runs in parallel on thousands of nodes, processing PB of data simultaneously.

The Shuffle Phase: Grouping

MAP OUTPUT

Node 1

`(apple, 1)` `(banana, 1)`

Node 2

`(apple, 2)` `(banana, 2)`

Partition by Term

Hash(TermID) % R
determines destination

REDUCE INPUT

Node 3 (A-M)

`(apple, [1, 2])`

Sorted by DocID

Node 4 (N-Z)

`(banana, [1, 2])`

Sorted by DocID

Performance Bottleneck:
The Shuffle phase involves moving massive amounts of data across the network. It is typically the most expensive part of the pipeline.

The Reduce Phase: Aggregation

1. Receive Grouped Data

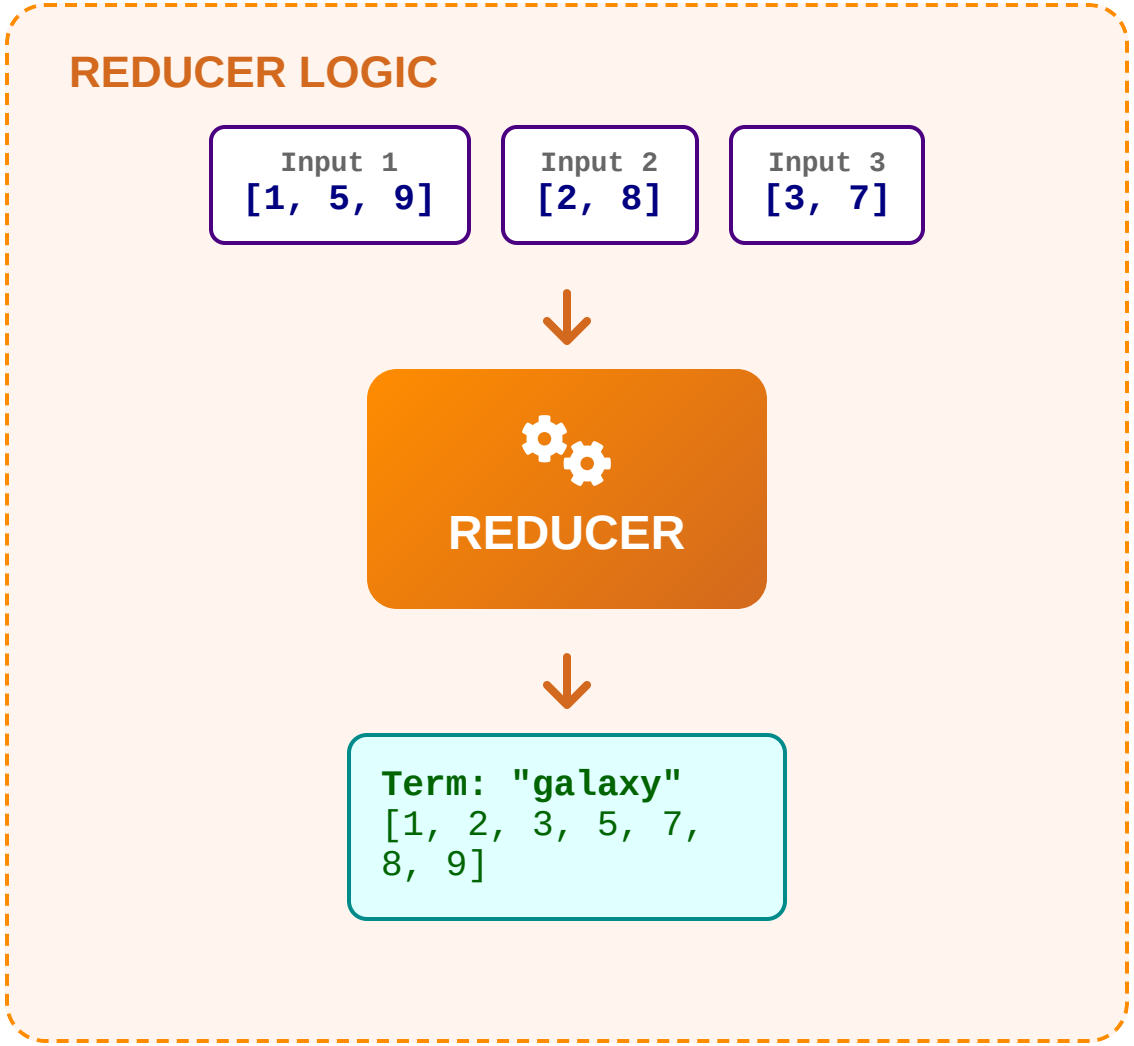
The Reducer receives all values associated with a specific key (Term).
Input: (Term, [List of DocIDs])

2. Merge & Sort

Combines DocIDs from different map tasks into a single list.
Sorts DocIDs to allow efficient compression later.

3. Write Final Index

Writes the final **Postings List** to the distributed file system (GFS/HDFS).



Partitioning Strategies

TERM PARTITIONING

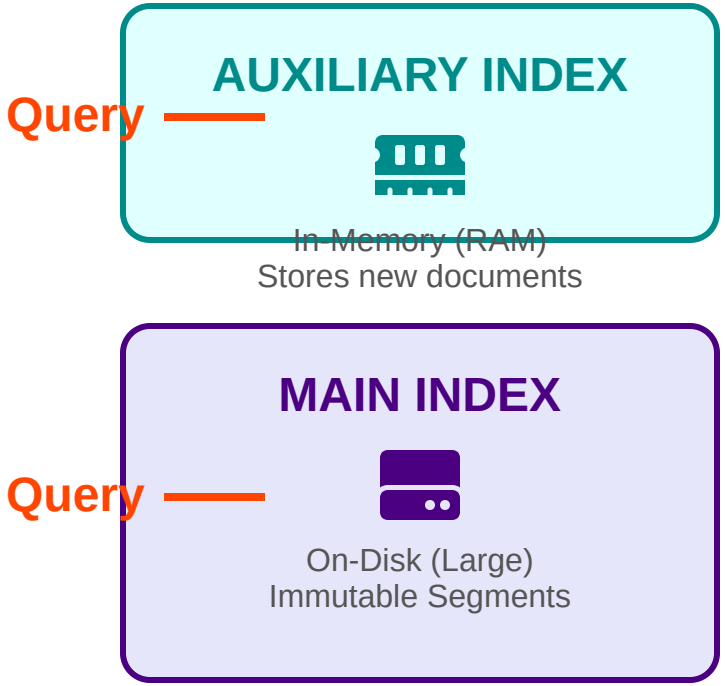
- **Global Index:** Terms A-M on Node 1, N-Z on Node 2.
- ✓ Good for multi-word queries (often on same node).
- ✗ **Hot Spot Issue:** Common terms overload specific nodes.

DOCUMENT PARTITIONING

- **Local Index:** Docs 1-1000 on Node 1, 1001-2000 on Node 2.
- ✓ **Scalable:** Easy to add new nodes. Even load distribution.
- ⚠ **Scatter-Gather:** Query must be sent to ALL nodes.

Industry Standard: **Document Partitioning**(Used by Google, Elasticsearch, Solr)

Dynamic Indexing: Handling Updates



New documents are added to the small **Auxiliary Index** in memory. This is fast ($O(1)$).

2. Search

Queries are sent to **BOTH** indexes. Results are merged on the fly.
Result = Main U Aux

3. Deletions

Use a **Bit Vector** to mark deleted DocIDs. Filter these out during search results merging.

4. Periodic Merge

When Aux is full, merge it into Main. (Logarithmic Merge strategy avoids re-writing everything often).

Logarithmic Merge (LSM Trees)

Write-Optimized

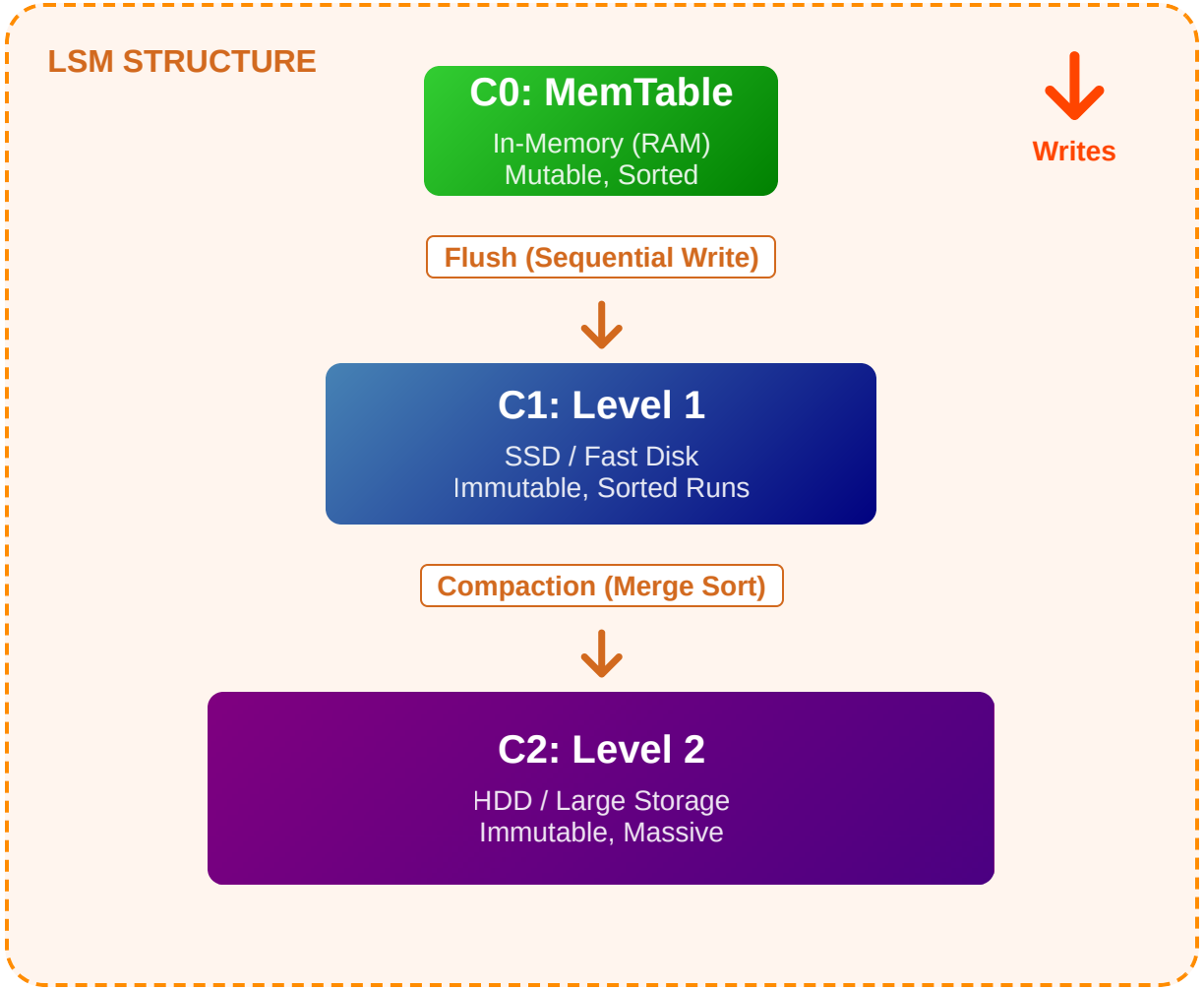
Writes are always appended to the in-memory buffer (MemTable). No random disk I/O means extremely high throughput.

Immutable Segments

Once written to disk, files (SSTables) are never modified. This simplifies concurrency and crash recovery.

Background Merge

Smaller segments are periodically merged into larger ones (Compaction) to keep read costs low and reclaim space.



Index Compression: Why & How

Compression is not just about saving disk space. In Information Retrieval, it is a critical performance optimization.

DISK SPACE

Reduces the physical footprint of the index, lowering storage costs.

I/O SPEED

Reading compressed data from disk is faster than reading raw data, even with decompression overhead.

CACHING (RAM)

The Big Win: A compressed index is more likely to fit entirely in memory (OS Cache), avoiding slow disk seeks.

Requirement: Lossless Compression

Unlike images or audio, IR indexes require **lossless** compression. We must be able to reconstruct the exact original text information.

Dictionary Compression

Goal: Keep Dictionary in Main Memory (RAM)

Disk seeks are slow. If the dictionary fits in RAM, we can look up terms instantly. But vocabulary grows with collection size (Heaps' Law).

Approach 1: Fixed Width

Allocating 20 bytes per term:

c	a	t	_	_	_
a	u	t	o	_	_

- ✗ **Wasteful:** Short terms waste ~60% space.
- ✗ **Limited:** Can't store terms > 20 chars.

Approach 2: String Array

Single continuous string + Pointers:

```
catautomation...
```

ptr:0 ptr:3 ptr:13

- ✓ **Compact:** No wasted space.
- ! **Overhead:** Need 4 bytes for term pointer.

Front Coding: Prefix Compression

INPUT: SORTED DICTIONARY TERMS

automata automate automatic automation

OUTPUT: COMPRESSED STRING

8automata ◊ 7e ◊ 7ic ◊ 7ion

Result: We store the prefix "automat" (7 bytes) only once implicitly, saving ~21 bytes in this small block.

Postings Compression: Gap Encoding

1. RAW DOCUMENT IDS (SORT

45 - 33 = 12 | 52 - 45 = 7 | 90 - 52 = 38 | 120 - 90 = 30

33	45	52	90	120

2. GAP ENCODED LIST (D-GAPS)

33	12	7	38	30

Why do this?

For frequent terms, gaps are very small (often < 255). Small numbers require fewer bits to store than large, raw DocIDs (which can be millions).

Variable Byte (VB) Encoding

A simple, byte-aligned compression scheme. It uses an integral number of bytes to encode a gap, depending on the gap size.

THE ANATOMY OF A VB BYTE (8 BITS)

1	0 1 0 1 1 0 1
C-BIT 1 Bit	PAYLOAD 7 Bits (Data)

Continuation Bit = 1

Indicates this is the **LAST** byte of the number. Stop reading and decode.

→ Continuation Bit = 0

Indicates **MORE** bytes follow. Read the next byte and append its payload.

VB Encoding: Step-by-Step

1

TARGET INTEGER (GAP)

824 (Binary: 11 0011 1000)

↓

2

SPLIT INTO 7-BIT GROUPS (RIGHT TO LEFT)

Group 2 (Pad) | Group 1 (Low 7)

`0000110` `0111000`

↓

3

ADD CONTINUATION BIT (C-BIT)

c=0 (More) | c=1 (End)

`00000110` `10111000`

↓

FINAL BYTE STREAM (HEX)

Gamma Encoding

Bit-Level Compression: Unlike Variable Byte (which wastes bits to align to 8-bit boundaries), Gamma encoding uses exactly as many bits as needed.

LENGTH (UNARY)
1110

+

OFFSET (BINARY)
010

1. Write gap G in binary.
2. Remove leading '1'. This is the **Offset**.
3. Write length of Offset in **Unary** (1s then 0).

Gap (G)	Binary	Offset	Length	Gamma Code (Unary + Offset)
1	1	(empty)	0	**0**
3	11	1	1	**101**
9	1001	001	3	**1110001**
13	1101	101	3	**1110101**

Gamma Encoding: Example

1. NUMBER	13
2. BINARY	1101
3. SPLIT	1 101 (Offset)
4. LENGTH	Length of Offset = 3
5. UNARY	Unary(3) = 1110

FINAL GAMMA CODE

1110101

Trade-offs: Variable Byte vs. Gamma

Choosing a compression algorithm is a balance between **space savings** (disk/RAM) and **decoding speed** (CPU).

METRIC	VARIABLE BYTE (VB)	GAMMA ENCODING
Alignment	✓ **Byte-Aligned (Simple)**	Bit-Aligned (Complex)
Compression Ratio	Good (~50%)	✓ **Excellent (~30-40%)**
Decoding Speed	⚡ **Very Fast**	Slow (Bit manipulation)
Implementation	☺ **Simple**	Complex

The Verdict

In modern search engines (like Lucene/Elasticsearch), **Decoding Speed** is usually the bottleneck, not disk space. Therefore, **Variable Byte** (or SIMD variants like PForDelta) is generally preferred over Gamma.

Modern Compression: PForDelta

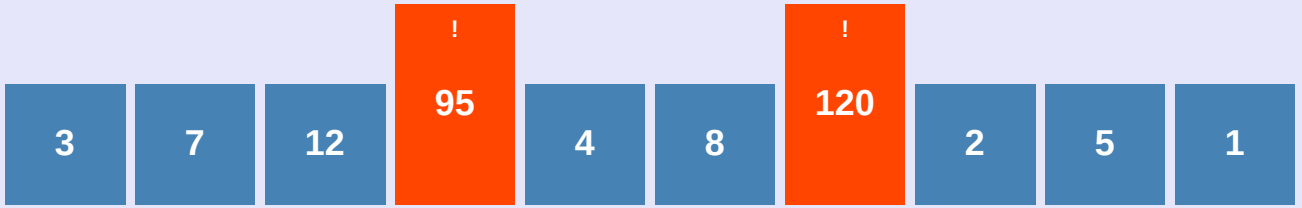
THE PROBLEM

Variable Byte encoding requires checking a continuation bit for *every byte*. This causes **branch mispredictions** in modern CPUs, stalling the pipeline.

THE SOLUTION

Use a **fixed bit-width** (e.g., 5 bits) for a block of 128 numbers. This allows the CPU to process them in parallel.

PATCHED FRAME OF REFERENCE (PFOR)



Normal (90%): Fit in b bits. Packed tightly.

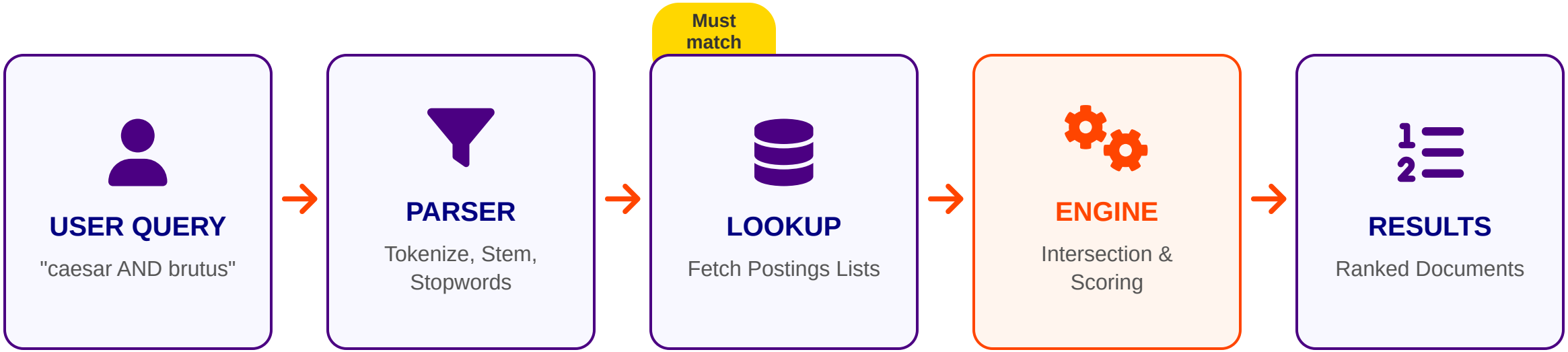
Exceptions (10%): Too large. Stored separately ("Patched").

SIMD Optimization (AVX-512)

Because the bit-width is fixed, we can use **Single Instruction, Multiple Data** instructions to decode 128 integers in a single CPU cycle.

5X FASTER THAN VBYTE

Query Processing: The Pipeline

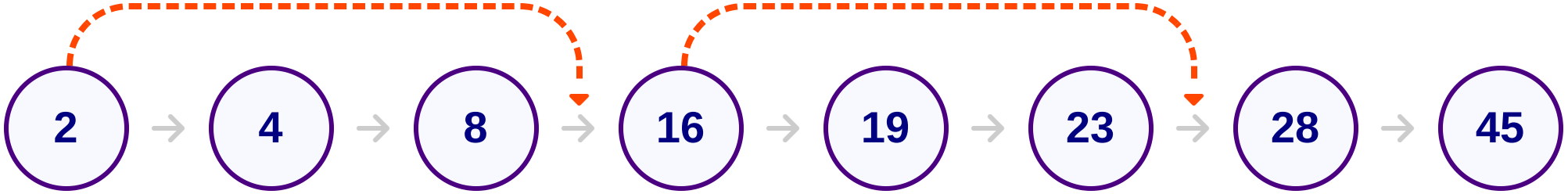


The Latency Constraint

Users expect results in **sub-second** time (< 200ms). The "Engine" step (Intersection) is the bottleneck and must be highly optimized (e.g., Skip Pointers).

Skip Pointers

Standard intersection of two sorted lists takes $O(M+N)$ time. By adding "skip" pointers, we can jump over non-matching documents to accelerate the process.



Placement Heuristic

For a postings list of length L, place skip pointers every $\sqrt{L}$ items. This balances the cost of skipping vs. checking.

When to Skip?

If `skip_ptr.docID ≤ current_other_docID`, take the skip. Otherwise, follow the regular `next` pointer.

Phrase Queries: The Problem of Order

Bag of Words Model

Standard inverted indexes store **sets** of words, ignoring their order.

To an index, these are identical:

- "The **New York** Times"
- "The **York New** Times"

⚠️ **Consequence:** A query for "New York" might return documents about "York New" (False Positives).

1. Bi-word Indexes

Index every consecutive pair of terms as a single token.
Example: "new york", "york times"

2. Positional Indexes

Store the exact position of every term occurrence in the postings list.
Example: new: [1, 5, 20], york: [2, 6, 21]

Bi-word Indexes

ORIGINAL TEXT

New | York | University

→

INDEX TERMS (VOCABULARY)

new york | york university

Advantages

- ✓ **Fast Retrieval:** Phrase queries become standard Boolean AND queries. No position checking needed.
- ✓ **Simple Implementation:** Works with any standard inverted index engine.

Disadvantages

- × **Index Blowup:** Vocabulary size can increase significantly (though not quite V^2 in practice).
- × **False Positives:** "Stanford University Palo Alto" matches query "University Palo" even if semantically unrelated.

**Often restricted to "Noun Phrases" (e.g., "High School") to reduce index size.*

Positional Indexes: Precision at a Cost

Doc 1:

1	2	3	4	5	6
to	be	or	not	to	be

```
to: 1: [1, 5]
```

```
be: 1: [2, 6]
```

Advantages

- **Exact Phrase Matching:** No false positives like "York New".
- **Proximity Search:** "Gates" WITHIN 5 WORDS OF "Microsoft".

Disadvantages

- **Storage Cost:** Indexes are 2-4x larger than non-positional ones.
- **Processing Cost:** Merging requires checking positions, not just DocIDs.

Positional Merge Algorithm

```
function PositionalIntersect(p1, p2, k):
answer = [] while p1 != NIL and p2 != NIL:
if p1.docID == p2.docID: l = [] pp1 =
p1.positions pp2 = p2.positions // Check
proximity for this doc while pp1 != NIL:
while pp2 != NIL: if |pp1.pos - pp2.pos| <=
k: l.add(pp2.pos) else if pp2.pos > pp1.pos:
break pp2 = pp2.next pp1 = pp1.next if l is
not empty: answer.add(p1.docID, l) p1 =
p1.next; p2 = p2.next else if p1.docID <
p2.docID: p1 = p1.next else: p2 = p2.next
return answer
```

Execution Trace (k=1)

"to":	2	10	25
"be":	3	15	26

Comparisons:

- $|2 - 3| = 1 \leq 1$ ✓ Match
- $|10 - 15| = 5 > 1$ ✗ Fail
- $|25 - 26| = 1 \leq 1$ ✓ Match

🏆 **Result: Doc 4 matches at [3, 26]**

Wildcard Queries

→ Trailing Wildcard

`comput*`

Finds: *computer, computing, computation*.
Solution: Standard B-tree lookup.

← Leading Wildcard

`*mon`

Finds: *lemon, demon, pokemon*.
Solution: Reverse B-tree (index "nomel").

↔ General Wildcard

B-TREE RANGE LOOKUP

```
...
compromise
compulsion
computation
compute
computer
computing
comrade
...
```

For `comput*`, we walk the B-tree to find the first term starting with "comput", then retrieve all subsequent terms until the

Performance Cost: A query like `s*` is dangerous! It retrieves all terms starting with 's' (potentially 10% of the dictionary), leading to a massive OR query.

Permuterm Index: Handling Wildcards

🔁 The Idea

To handle wildcard queries like `X*Y` efficiently, we map them to a standard **prefix query**.

$ The Mechanism

1. Append a special character `$` to the term.
2. Store **all rotations** of the term in the B-tree dictionary.
3. Each rotation points to the original term.

Trade-off: Quadruples the dictionary size (roughly), but enables fast $O(\log N)$ lookup.

Example: "hello"

hello		$
ello	$	h
llo	$	he
lo	$	hel
o	$	hell

↓

Query: "h*o" (starts with h, ends with o)

```
1. Rotate to move * to end:
o$h*

2. Lookup:
B-tree range search for o$h
```

k-gram Indexes

Definition

A **k-gram** is a sequence of k characters. We augment terms with $ to mark boundaries.

`$castle$` → `$ca` `cas` `ast` `stl` `tle` `le$`

The Index (k=3)

etr	`metric, petrify, retrieval`
ret	`retrieval, retina, return`
val	`festival, retrieval, value`

QUERY: "RE*VE"

1. Extract k-grams: `$re`, `ve$`
2. Lookup terms containing `$re` AND `ve$`.
3. Result: *relive, remove, receive, retrieve*.

Post-Filtering Required: *Must check if "re" actually precedes "ve" in the term (e.g., "verve" matches the k-grams but fails the wildcard).*

Spelling Correction

Two main principles: **Edit Distance** (transformations) and **k-gram Overlap** (shared substrings).

Edit Distance

Minimum number of operations (Insert, Delete, Replace) to transform string s_1 into s_2.

EXAMPLE: FAST → CATS

f → c	REPLACE
a → a	MATCH
s → t	REPLACE
t → s	REPLACE

Distance = 3

k-gram Overlap

Measure similarity by the number of shared k-grams (substrings of length k).

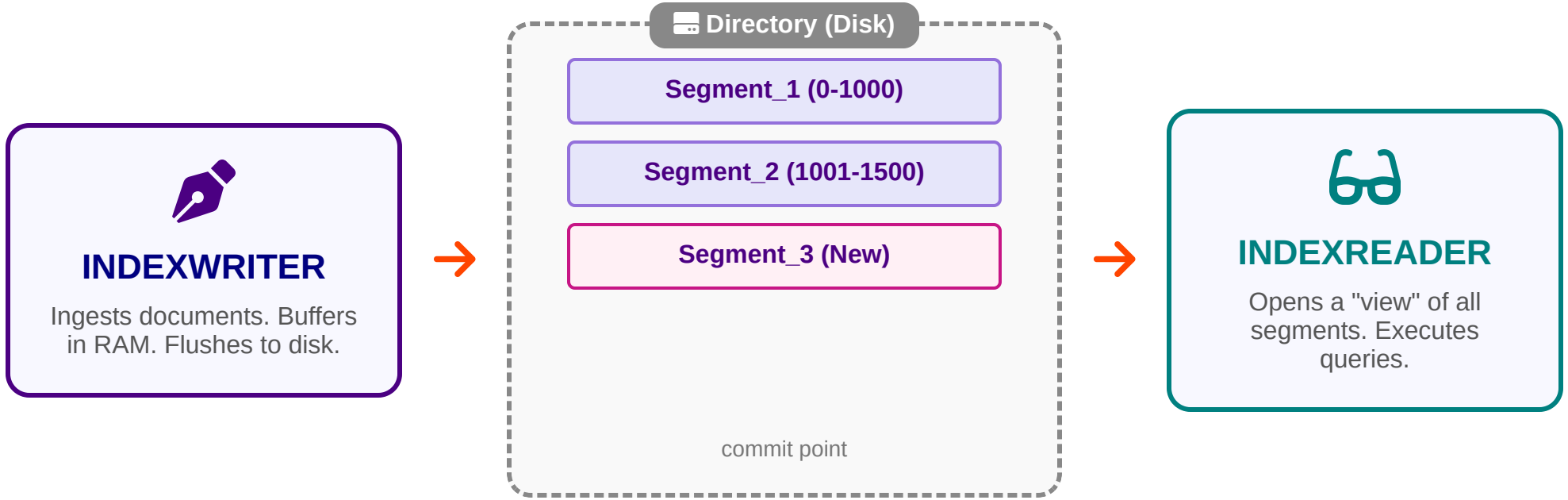
EXAMPLE: BORD VS. BOARD (2-GRAMS)

bord: {bo, or, rd}

board: {bo, oa, ar, rd}

Jaccard = 2 / 5 = 0.4

Real-World: Lucene Architecture



The Segment

A **Segment** is a complete, self-contained inverted index. Once written, it is **immutable** (never changed).

Near Real-Time (NRT)

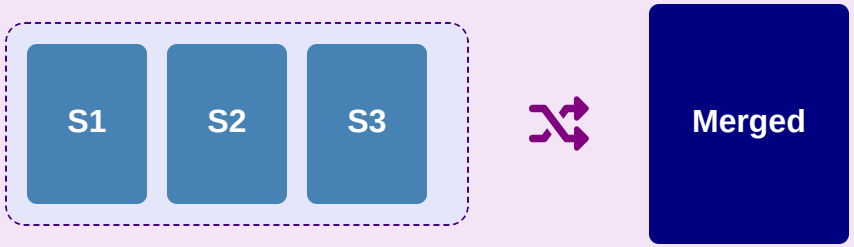
New documents are written to a new segment. The reader re-opens to see the new segment without restarting.

Lucene Segments: The Immutable Truth

1. THE WRITE PATH

Immutability: Once a segment is written to disk, it is *never modified*. This eliminates the need for complex read-locking, making search extremely fast and concurrent.

2. THE MERGE PATH



Background Merging: Too many small segments slow down search. Background threads continuously merge them into larger segments (TieredMergePolicy) to maintain performance.

Handling Deletes: Since segments are immutable, we can't remove data. Instead, we write a "Tombstone" to a separate `.liv` bitset. The data is physically removed only during a **merge**.

Real-World: Elasticsearch

THE ANALOGY

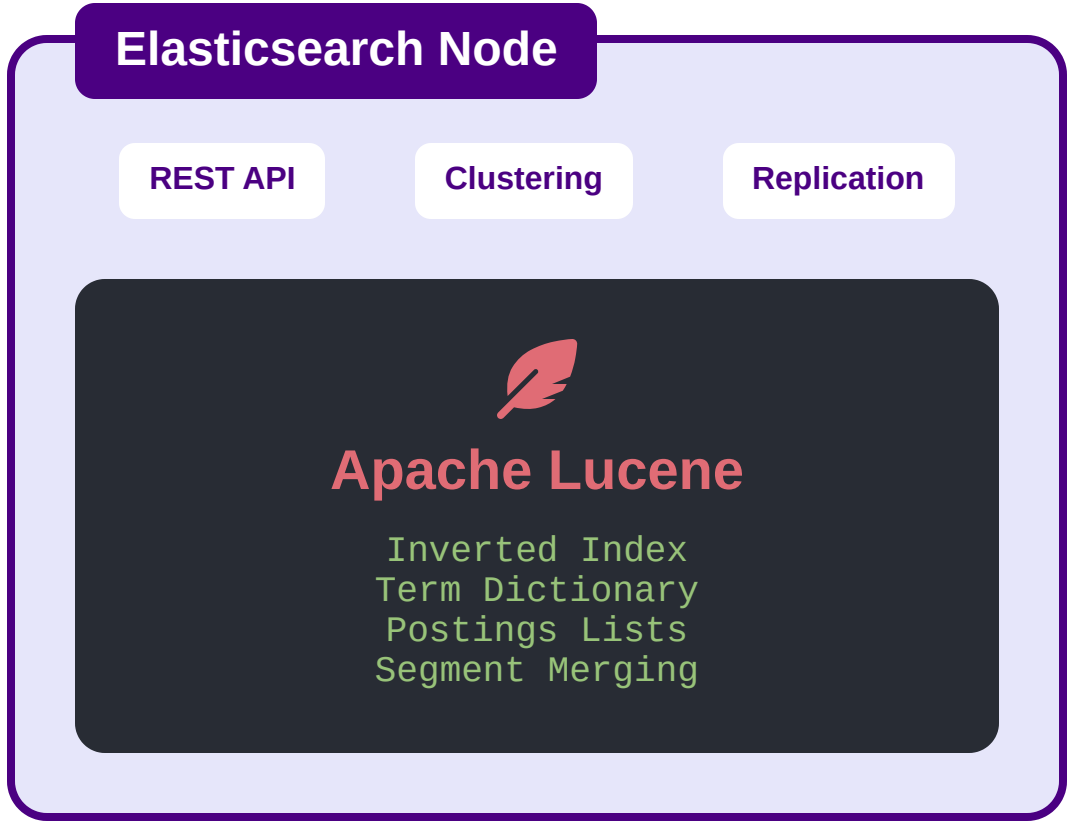
Lucene is the engine.
Elasticsearch is the car.

Lucene provides the core indexing libraries, but Elasticsearch makes it a distributed, usable server.

Distributed: Scales horizontally across hundreds of nodes.

RESTful API: Speak JSON over HTTP (PUT, GET, POST).

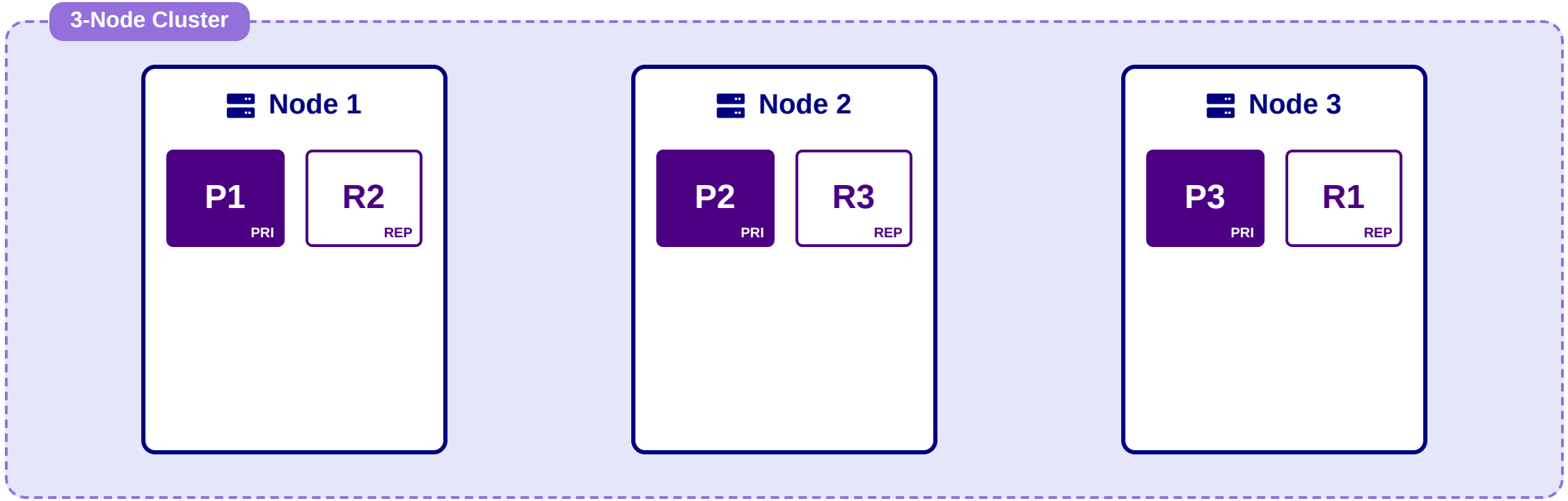
Near Real-Time: Documents are searchable within ~1 second.



Elasticsearch Sharding & Replication

INDEX (LOGICAL)
A collection of documents (e.g., "products").

SHARD (PHYSICAL)
A single Lucene instance. An index is split into N shards.



Horizontal Scaling: Add more nodes, and shards automatically rebalance.

High Availability: If Node 1 fails, R1 on Node 3 is promoted to Primary.

Elasticsearch Cluster Architecture

Master Node

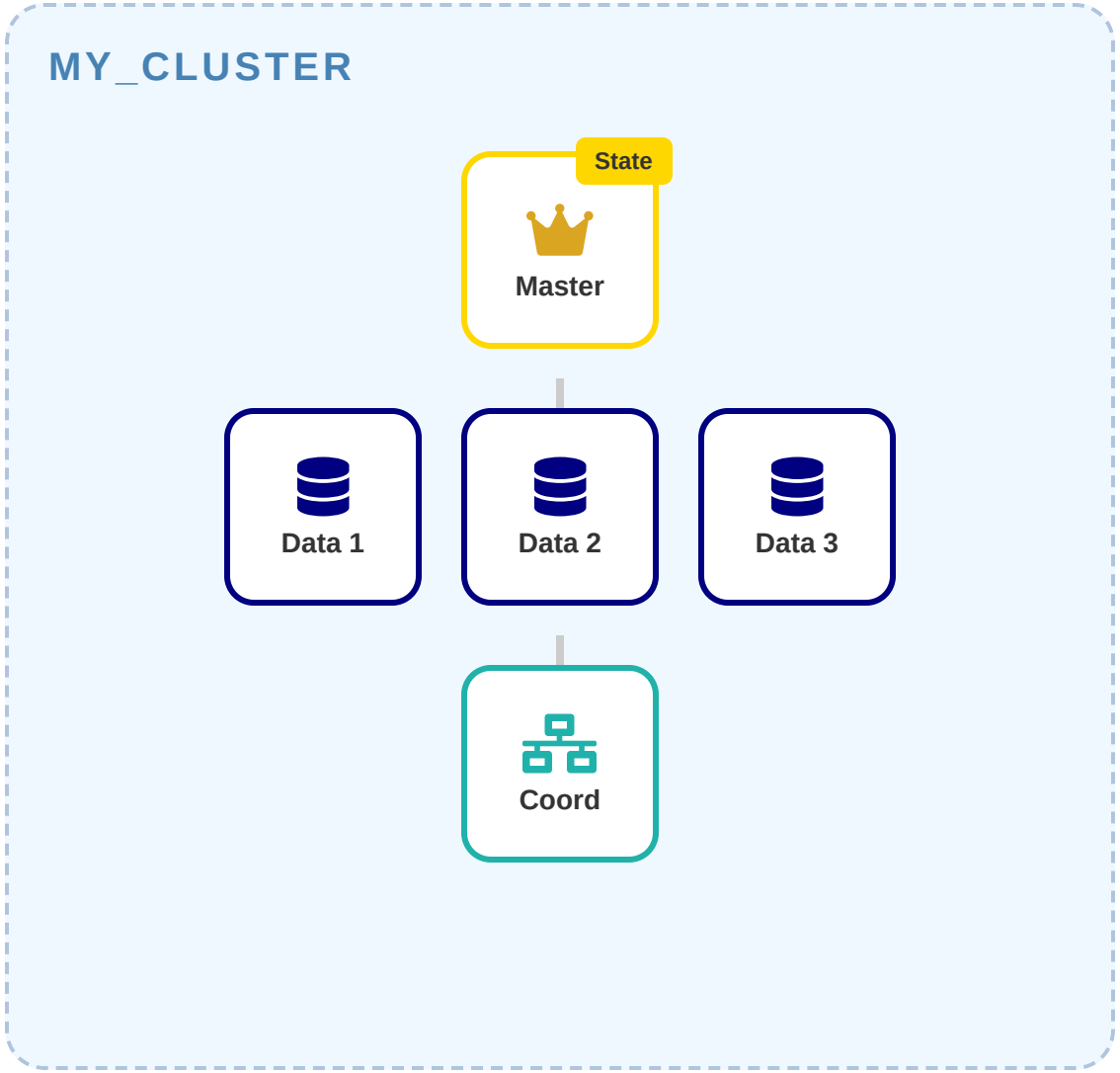
The "Brain". Manages the **Cluster State** (indices, mappings, settings). Only one active master at a time.

Data Node

The "Worker". Holds shards (data). Executes CRUD, search, and aggregations. Heavy I/O, Memory, and CPU usage.

Coordinating Node

The "Router". Receives client requests, routes them to data nodes, and aggregates the results (Scatter/Gather).



Case Study: Twitter Search (Earlybird)

The Challenge

- **High Velocity:** Thousands of tweets per second (TPS).
- **Real-Time:** Tweets must be searchable within seconds of posting.
- **Recency:** Users care about the *newest* content, not just relevance.

The Solution

- **In-Memory Index:** Active segment lives entirely in RAM for speed.
- **Lock-Free Writes:** Single-writer model avoids contention.
- **Time-Ordered:** Postings lists are sorted by Time, enabling fast reverse-chronological retrieval.

EARLYBIRD PIPELINE

Performance Optimization

Bulk Indexing

Never index documents one by one. Use **Bulk APIs** to send batches (e.g., 5MB - 15MB chunks). This drastically reduces network overhead and transaction costs.

RAM Buffer

Maximize the indexing buffer size. Larger buffers create **larger initial segments**, reducing the frequency and cost of future merges.

Storage (SSD vs HDD)

Search and merging involve heavy **Random I/O**. SSDs are essential for performance. Avoid NAS or remote storage if possible.

Merge Policy

Tune the merge factor. **Less merging** = faster indexing but slower search (more segments to check). Balance based on your use case (Write-heavy vs. Read-heavy).

Common Pitfalls in Indexing

MEMORY OVERFLOW

THE MISTAKE

Attempting to hold the entire dictionary or too many postings in RAM during construction.

THE FIX

Use **SPIMI** to write blocks to disk when RAM is full, then merge.

BAD BLOCK SIZING

THE MISTAKE

Blocks too small (high metadata overhead) or too large (slow decompression).

THE FIX

Tune block size to match disk I/O (e.g., 4KB - 64KB).

OVER-COMPRESSION

THE MISTAKE

Using complex algorithms (like Huffman) that save space but kill CPU performance.

THE FIX

Prefer **byte-aligned** codes (VByte) for faster decoding speed.

Advanced: Finite State Transducers (FST)

What is an FST?

A graph data structure that maps a **Term** (input) to a **Value** (output). Unlike a Trie, it shares both prefixes *and* suffixes.

Massive Compression: Reduces memory usage by 10x-100x compared to a HashMap.

In-Memory Index: Allows Lucene to hold the entire Term Dictionary in RAM for blazing speed.

FST STRUCTURE

moth	→ 5
mother	→ 9

Advanced: Roaring Bitmaps

Standard Bitsets are fast but sparse (waste RAM). Arrays are compact but slow.
Roaring Bitmaps use a hybrid approach to get the best of both worlds.

High 16 Bits — THE KEY (CHUNK ID)

Low 16 Bits — THE VALUE

↓

ARRAY CONTAINER

Used for **Sparse** chunks (< 4096 integers).
Stores a simple sorted list of 16-bit integers.

BITMAP CONTAINER

Used for **Dense** chunks (> 4096 integers).
Stores a fixed 8KB bitset (bitmap).

RUN CONTAINER

Used for **Sequences** (e.g., 10, 11, 12...).
Stores pairs of (Start, Length).

Lab: Building a Simple Indexer

indexer.py

```python
def index_document(doc_id, text, index):
    # 1. Tokenize by whitespace
    tokens = text.split()

    for token in tokens:
        # 2. Normalize (lower case)
        term = token.lower()

        # 3. Update Postings List
        if term not in index:
            index[term] = []

        if doc_id not in index[term]:
            index[term].append(doc_id)
```

1 Tokenization

Splits the raw text into individual words. In this simple example, we split by spaces.

2 Normalization

Converts tokens to a standard format (lowercase) so "Apple" and "apple" match.

3 Update Postings

Adds the current `doc_id` to the term's list. Checks for duplicates to keep the list sorted/unique.

RESULTING INDEX STRUCTURE

```
{
  "cat": [1, 4],
  "sat": [1],
  "mat": [2]
}
```

Lab: Compression Challenge

THE TASK

Implement the vb_encode function. It should take an integer ID and return a list of bytes (Variable Byte encoded).

Example: 824 (Binary: 1100111000)

Split: 0000110 | 0111000

↓

Add MSB: 00000110 | 10111000

↓

Hex: 0x06, 0xB8

vb_encode.py

```
def vb_encode(number): bytes_list = [] # 1.
Handle the last byte (MSB = 1)
bytes_list.append(number % 128 + 128) number //=
128 # 2. Handle remaining bytes (MSB = 0) while
number > 0: bytes_list.insert(0, number % 128)
number //= 128 return bytes_list
```

```
> vb_encode(824)
[6, 184] # 0x06, 0xB8
```

Summary of Week 4

Index Construction

- **BSBI:** Block Sort-Based Indexing for disk-based sorting.
- **SPIMI:** Single-Pass In-Memory Indexing for efficiency.
- **Dynamic:** Logarithmic merge for handling updates.

Distributed Indexing

- **MapReduce:** Framework for scaling.
- **Map:** Parses docs into (term, docID) pairs.
- **Reduce:** Inverts pairs into postings lists.

Index Compression

- **Dictionary:** Front Coding to save RAM.
- **Postings:** Variable Byte & Gamma codes.
- **Goal:** Fit more index in memory = Faster search.

Advanced Querying

- **Skip Pointers:** Faster intersection ($O(\sqrt{L})$).
- **Phrase Queries:** Bi-word & Positional indexes.
- **Wildcards:** Permuterm & k-gram indexes.

Key Takeaway: Efficient indexing balances construction speed, storage size, and query

Further Reading

Textbooks

Introduction to Information Retrieval

Manning, Raghavan, and Schütze (2008)

Chapter 4: Index Construction

Chapter 5: Index Compression

Managing Gigabytes

Witten, Moffat, and Bell (1999)

Chapter 3: Indexing

Chapter 4: Compression

Modern Information Retrieval

Baeza-Yates and Ribeiro-Neto (2011)

Chapter 8: Indexing and Searching

Key Research Papers

MapReduce: Simplified Data Processing on Large Clusters

Dean and Ghemawat (OSDI 2004)

The foundation of distributed indexing.

Inverted Files for Text Search Engines

Zobel and Moffat (ACM Computing Surveys 2006)

Comprehensive survey of indexing techniques.

Compression of Inverted Indexes For Fast Query Evaluation

Scholer et al. (SIGIR 2002)

Analysis of compression performance.

Review Questions

1. **ALGORITHMS**

   Compare **BSBI** and **SPIMI**. Why is SPIMI generally preferred for large-scale web indexing?

2. **COMPRESSION**

   Encode the integer **130** using **Variable Byte** code. Show the binary steps and the final hex value.

3. **DISTRIBUTED INDEXING**

   Explain the role of the **Master Node** in MapReduce. How does it handle node failures?

4. **QUERY OPTIMIZATION**

   Under what specific condition (document frequency ratio) are **Skip Pointers** most effective for intersection?

5. **WILDCARDS**